

FINETUNING_004

Stain-Robust Pathology Classifier

Fine-tuned ViT-L/16 on AMD MI300X

Dataset:	NCT-CRC-HE-100K (100 K patches) + CRC-VAL-HE-7K (7 K patches)
Architecture:	ViT-L/16 (307 M params), full fine-tune, bfloat16, torch.compile
Hardware:	Single AMD MI300X GPU (192 GB HBM3)
Clean acc.:	97.3 %
OOD acc.:	84.4 % (synthetically stain-shifted validation set)
Robustness:	3.4x smaller accuracy drop vs baseline (44 pp → 12.9 pp)
Models:	ResNet-50, ViT-B/16, ViT-L/16, Phikon, ViT-L/16 no-aug ablation

Notebooks: [shared.ipynb](#) · [00_environment_check.ipynb](#) · [01_data.ipynb](#) · [02_train.ipynb](#) · [03_evaluate_explain.ipynb](#) · [04_gradio_demo.ipynb](#) · [05_showcase.ipynb](#)

Shared Library (shared.ipynb)

`shared.ipynb` ? Core library (Phase A + B + helpers)

Run with `%run shared.ipynb` from the other notebooks. Defines, with no side effects:

```
* **Phase A** ? `PathologyPatchDataset`, Albumentations builders, `HEDStainJitter`,
  `MacenkoNormalizer`, `ensure_dataset`, `make_data loaders`.
* **Phase B** ? `build_model` (timm full fine-tune **and** frozen-Phikon + MLP head),
  `get_param_groups`.
* **Helpers** ? Grad-CAM (`GradCAMViT`/`GradCAMResNet`), metrics, resume-safe checkpoints.
```

This file deliberately does ****not**** download data or build models on import.

```
# Dependency bootstrap. The container env is not persistent, so this reinstalls each session.
import importlib.util, subprocess, sys

def _ensure(mod, pkg=None):
    "Install pkg (pip name) only if module `mod` is missing. Safe to call repeatedly."
    if importlib.util.find_spec(mod) is None:
        print(f"installing {pkg or mod} ...")
        subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                        "--root-user-action=ignore", pkg or mod], check=False)

# Core libs needed to import this file and train with timm + W&B:
for _mod, _pkg in {"albumentations": "albumentations", "skimage": "scikit-image",
                  "cv2": "opencv-python-headless", "timm": "timm",
                  "sklearn": "scikit-learn", "seaborn": "seaborn",
                  "matplotlib": "matplotlib", "wandb": "wandb"}.items():
    _ensure(_mod, _pkg)

# transformers (Phikon), gradio (demo) and torchstain (Macenko) are installed on demand by
# the code that needs them, to avoid pulling heavy/conflicting deps during plain training.
```

```

import os, sys, json, math, time, random, logging, zipfile, urllib.request
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("ft004")

# ---- constants -----
CLASSES = ["ADI", "BACK", "DEB", "LYM", "MUC", "MUS", "NORM", "STR", "TUM"]
CLASS_DESCRIPTIONS = {
    "ADI": "Adipose (fat tissue)",
    "BACK": "Background",
    "DEB": "Debris",
    "LYM": "Lymphocytes",
    "MUC": "Mucus",
    "MUS": "Smooth muscle",
    "NORM": "Normal colon mucosa",
    "STR": "Cancer-associated stroma",
    "TUM": "Colorectal adenocarcinoma epithelium",
}
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)

def get_device():
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")

def get_amp_dtype(device):
    '''bf16 on MI300X (native, no GradScaler); fp16 on other CUDA; fp32 on CPU.'''
    if device.type != "cuda":
        return torch.float32
    if torch.cuda.is_bf16_supported():
        return torch.bfloat16
    return torch.float16

```

Phase A ? Augmentation (Albumentations)

`HEDStainJitter` is the stain-robustness innovation: it perturbs the Hematoxylin?Eosin?DAB colour space (Tellez et al. 2018) to simulate inter-lab / inter-scanner stain variation ? the #1 deployment failure mode. Spatial transforms force the model onto morphology, not colour.

```

import albumentations as A
from albumentations.pytorch import ToTensorV2
import cv2
from skimage.color import rgb2hed, hed2rgb

class HEDStainJitter(A.ImageOnlyTransform):
    '''Randomly perturb the H&E (HED) colour space. img: HxWx3 uint8 RGB.'''

    def __init__(self, theta=0.05, p=0.5):
        super().__init__(p=p)
        self.theta = theta

    def apply(self, img, **params):
        rgb = img.astype(np.float32) / 255.0
        hed = rgb2hed(rgb)
        for c in range(3):
            alpha = np.random.uniform(1 - self.theta, 1 + self.theta)
            beta = np.random.uniform(-self.theta, self.theta)
            hed[:, :, c] = hed[:, :, c] * alpha + beta
        out = np.clip(hed2rgb(hed), 0.0, 1.0)
        return (out * 255.0).astype(np.uint8)

    def get_transform_init_args_names(self):
        return ("theta",)

def build_train_aug(use_stain_aug=True, img_size=224, theta=0.05):
    tfms = [
        A.RandomRotate90(p=0.5),
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.5),
        A.Affine(translate_percent=0.05, scale=(0.9, 1.1), rotate=(-15, 15),
            mode=cv2.BORDER_REFLECT_101, p=0.5),
        A.OneOf([
            A.ElasticTransform(alpha=1.0, sigma=20.0, p=1.0),
            A.GridDistortion(num_steps=5, distort_limit=0.1, p=1.0),
            A.OpticalDistortion(distort_limit=0.1, p=1.0),
        ], p=0.3),
    ]
    if use_stain_aug:
        tfms += [
            HEDStainJitter(theta=theta, p=0.8),
            A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15, val_shift_limit=10, p=0.3),
            A.RandomBrightnessContrast(brightness_limit=0.1, contrast_limit=0.1, p=0.3),
        ]
    tfms += [
        A.Resize(img_size, img_size),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ToTensorV2(),
    ]
    return A.Compose(tfms)

def build_eval_aug(img_size=224):
    return A.Compose([
        A.Resize(img_size, img_size),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ToTensorV2(),
    ])

```

```
def build_ood_aug(img_size=224, theta=0.15):
    '''Strong synthetic stain shift = a 'different lab' for the OOD ablation.'''
    return A.Compose([
        HEDStainJitter(theta=theta, p=1.0),
        A.HueSaturationValue(hue_shift_limit=25, sat_shift_limit=30, val_shift_limit=20, p=1.0),
        A.Resize(img_size, img_size),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ToTensorV2(),
    ])
```

```
class MacenkoNormalizer:
    '''Optional Macenko stain normalization (torchstain). For OOD targets / demo uploads.'''

    def __init__(self, target_img=None):
        _ensure("torchstain")
        import torchstain
        self.normalizer = torchstain.normalizers.MacenkoNormalizer(backend="numpy")
        self.fitted = False
        if target_img is not None:
            self.fit(target_img)

    def fit(self, target_img): # target_img: HxWx3 uint8 RGB
        self.normalizer.fit(np.asarray(target_img, dtype=np.uint8))
        self.fitted = True

    def __call__(self, img): # img: HxWx3 uint8 RGB -> uint8 RGB
        if not self.fitted:
            return img
        try:
            norm, _, _ = self.normalizer.normalize(I=np.asarray(img, dtype=np.uint8), stains=False)
            return np.asarray(norm, dtype=np.uint8)
        except Exception as e:
            logger.warning(f"Macenko normalize failed; returning original ({e})")
            return img
```

Phase A ? Dataset & data acquisition

```

from PIL import Image

_IMG_EXTS = {".png", ".jpg", ".jpeg", ".tif", ".tiff", ".bmp"}

class PathologyPatchDataset(Dataset):
    '''Reads class-named subfolders (ImageFolder layout). Albumentations-based transforms.'''

    def __init__(self, root, classes=None, transform=None, macenko=None):
        self.root = Path(root)
        if classes is None:
            classes = sorted([d.name for d in self.root.iterdir() if d.is_dir()])
        self.classes = classes
        self.class_to_idx = {c: i for i, c in enumerate(classes)}
        self.samples = []
        for c in classes:
            cdir = self.root / c
            if not cdir.is_dir():
                logger.warning(f"Missing class dir: {cdir}")
                continue
            for f in sorted(cdir.iterdir()):
                if f.suffix.lower() in _IMG_EXTS:
                    self.samples.append((str(f), self.class_to_idx[c]))
        if not self.samples:
            raise RuntimeError(f"No images found under {self.root}")
        self.transform = transform
        self.macenko = macenko
        logger.info(f"Dataset {self.root.name}: {len(self.samples)} imgs / {len(classes)} classes")

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        try:
            img = np.array(Image.open(path).convert("RGB"))
        except Exception as e:
            logger.warning(f"Read failed {path} ({e}); skipping to next")
            return self.__getitem__((idx + 1) % len(self.samples))
        if self.macenko is not None:
            img = self.macenko(img)
        if self.transform is not None:
            img = self.transform(image=img)["image"]
        return img, label

```

```

# Zenodo record 1214456 = NCT-CRC-HE-100K (train) + CRC-VAL-HE-7K (patient-disjoint val)
ZENODO = {
    "NCT-CRC-HE-100K": "https://zenodo.org/records/1214456/files/NCT-CRC-HE-100K.zip?download=1",
    "CRC-VAL-HE-7K": "https://zenodo.org/records/1214456/files/CRC-VAL-HE-7K.zip?download=1",
}
# HuggingFace fallback (no token needed): use `datasets.load_dataset` mirrors if Zenodo is slow.

def _download(url, dest):
    logger.info(f"Downloading -> {dest}")
    last = [0]
    def hook(blocks, bs, total):
        if total > 0:
            pct = 100.0 * blocks * bs / total
            if pct - last[0] >= 2:
                last[0] = pct
                print(f"\r {pct:5.1f}%", end="", flush=True)
    urllib.request.urlretrieve(url, dest, reporthook=hook)
    print()

def _find_class_root(base, name):
    '''Return the dir that actually contains the 9 class subfolders.

    The search is scoped to base/name so that, e.g., a CRC-VAL-HE-7K lookup can never
    resolve to an already-extracted NCT-CRC-HE-100K folder (and vice versa).
    '''
    cand = base / name
    if cand.is_dir() and any((cand / c).is_dir() for c in CLASSES):
        return cand
    if cand.is_dir():
        for d in cand.rglob("*"):
            if d.is_dir() and sum((d / c).is_dir() for c in CLASSES) >= 5:
                return d
    return cand

def ensure_dataset(persist_dir, which=("NCT-CRC-HE-100K", "CRC-VAL-HE-7K"), delete_zip=True):
    persist_dir = Path(persist_dir)
    data_dir = persist_dir / "data"
    data_dir.mkdir(parents=True, exist_ok=True)
    out = {}
    for name in which:
        root = _find_class_root(data_dir, name)
        if root.is_dir() and any((root / c).is_dir() for c in CLASSES):
            logger.info(f"{name} already present: {root}")
            out[name] = root
            continue
        zip_path = data_dir / f"{name}.zip"
        if not zip_path.exists():
            try:
                _download(ZENODO[name], zip_path)
            except Exception as e:
                raise RuntimeError(f"Download failed for {name} ({e}). "
                                   f"Manually place the extracted folder at {data_dir / name}") from e
        logger.info(f"Extracting {zip_path.name} ...")
        with zipfile.ZipFile(zip_path) as z:
            z.extractall(data_dir / name)
        if delete_zip:
            zip_path.unlink(missing_ok=True) # respect the shared-folder size ceiling
        out[name] = _find_class_root(data_dir, name)
        logger.info(f"{name} ready: {out[name]}")

```

```
return out
```

```
def make_data loaders(train_dir, val_dir, cfg):
    train_ds = PathologyPatchDataset(
        train_dir,
        transform=build_train_aug(cfg["use_stain_aug"], cfg["img_size"], cfg["hed_theta"]),
    )
    val_ds = PathologyPatchDataset(
        val_dir, classes=train_ds.classes, transform=build_eval_aug(cfg["img_size"]),
    )
    common = dict(num_workers=cfg["num_workers"], pin_memory=cfg["pin_memory"])
    if cfg["num_workers"] > 0:
        common.update(persistent_workers=True, prefetch_factor=cfg.get("prefetch_factor", 4))
    train_loader = DataLoader(train_ds, batch_size=cfg["batch_size"], shuffle=True,
                             drop_last=True, **common)
    val_loader = DataLoader(val_ds, batch_size=cfg["eval_batch_size"], shuffle=False, **common)
    return train_loader, val_loader, train_ds.classes
```

Phase B ? Model backbone strategy

Config 1 (full fine-tune): `timm` ResNet-50 / ViT-B / ViT-L / Swin on ImageNet weights.

Uses PyTorch SDPA attention (the portable flash/efficient kernel on ROCm).

Config 2 (frozen FM + MLP head): `owkin/phikon` (un-gated) as a frozen feature extractor, CLS embedding -> trainable MLP head. Only the head is optimized.

```

TIMM_MAP = {
    "resnet50": "resnet50.a1_in1k",
    "vit_b":    "vit_base_patch16_224.augreg2_in21k_ft_in1k",
    "vit_l":    "vit_large_patch16_224.augreg_in21k_ft_in1k",
    "swin_b":   "swin_base_patch4_window7_224.ms_in22k_ft_in1k",
}

class PhikonClassifier(nn.Module):
    '''Frozen Phikon backbone + trainable MLP head (Config 2).'''

    def __init__(self, num_classes=9, hidden=512, dropout=0.3, model_id="owkin/phikon"):
        super().__init__()
        _ensure("transformers")
        from transformers import AutoModel
        try:
            self.backbone = AutoModel.from_pretrained(model_id, attn_implementation="sdpa")
        except Exception as e:
            logger.warning(f"sdpa attn unavailable ({e}); using default attention")
            self.backbone = AutoModel.from_pretrained(model_id)
        for p in self.backbone.parameters():
            p.requires_grad_(False)
        self.backbone.eval()
        feat = self.backbone.config.hidden_size
        self.head = nn.Sequential(
            nn.LayerNorm(feat), nn.Linear(feat, hidden), nn.GELU(),
            nn.Dropout(dropout), nn.Linear(hidden, num_classes),
        )

    def train(self, mode=True):
        super().train(mode)
        self.backbone.eval() # backbone stays frozen / eval
        return self

    def forward(self, x):
        with torch.no_grad():
            out = self.backbone(pixel_values=x)
            cls = out.last_hidden_state[:, 0]
        return self.head(cls)

def build_model(cfg, num_classes=9):
    name = cfg["model"]
    if name == "phikon":
        return PhikonClassifier(num_classes=num_classes,
                                model_id=cfg.get("phikon_id", "owkin/phikon"))
    import timm
    if name not in TIMM_MAP:
        raise ValueError(f"Unknown model '{name}'. Options: {list(TIMM_MAP) + ['phikon']}")
    return timm.create_model(TIMM_MAP[name], pretrained=cfg.get("pretrained", True),
                             num_classes=num_classes)

def get_param_groups(model, cfg):
    '''Layer-wise LR: head LR > backbone LR for ViT/Swin; single group for ResNet/Phikon.'''
    name, lr, wd = cfg["model"], cfg["lr"], cfg["weight_decay"]
    if name == "phikon":
        return [{"params": [p for p in model.parameters() if p.requires_grad],
                    "lr": lr, "weight_decay": wd}]
    head_keys = ("head", "fc", "classifier")
    head, backbone = [], []
    for n, p in model.named_parameters():

```

```

        if not p.requires_grad:
            continue
        (head if any(k in n for k in head_keys) else backbone).append(p)
    groups = []
    if head:
        groups.append({"params": head, "lr": lr, "weight_decay": wd})
    if backbone:
        groups.append({"params": backbone, "lr": lr * cfg.get("backbone_lr_mult", 0.1),
                        "weight_decay": wd})
    return groups

```

Helpers ? metrics, Grad-CAM, resume-safe checkpoints

```

from sklearn.metrics import (accuracy_score, f1_score, confusion_matrix,
                             classification_report)

@torch.no_grad()
def run_inference(model, loader, device, amp_dtype):
    model.eval()
    preds, labels = [], []
    use_amp = device.type == "cuda"
    for x, y in loader:
        x = x.to(device, non_blocking=True)
        if use_amp:
            with torch.autocast(device_type="cuda", dtype=amp_dtype):
                out = model(x)
        else:
            out = model(x)
        preds.append(out.argmax(1).detach().cpu())
        labels.append(y)
    return torch.cat(preds).numpy(), torch.cat(labels).numpy()

def compute_metrics(labels, preds, classes=CLASSES):
    return {
        "accuracy": float(accuracy_score(labels, preds) * 100),
        "macro_f1": float(f1_score(labels, preds, average="macro") * 100),
        "per_class_f1": {c: float(v * 100) for c, v in
                          zip(classes, f1_score(labels, preds, average=None))},
    }

def plot_confusion_matrix(labels, preds, classes=CLASSES, save_path=None, normalize=True):
    import matplotlib.pyplot as plt
    import seaborn as sns
    cm = confusion_matrix(labels, preds, labels=list(range(len(classes))))
    if normalize:
        cm = cm.astype(float) / cm.sum(axis=1, keepdims=True).clip(min=1)
    fig, ax = plt.subplots(figsize=(9, 7))
    sns.heatmap(cm, annot=True, fmt=".2f" if normalize else "d", cmap="Blues",
                xticklabels=classes, yticklabels=classes, ax=ax, cbar=True)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True")
    ax.set_title("Confusion Matrix ? CRC-VAL-HE-7K")
    plt.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=180, bbox_inches="tight")
        logger.info(f"Saved confusion matrix -> {save_path}")
    return fig

```

```

class _GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.acts = None
        self.grads = None
        self._h = [target_layer.register_forward_hook(self._fwd),
                    target_layer.register_full_backward_hook(self._bwd)]

    def _fwd(self, m, i, o):
        self.acts = o

    def _bwd(self, m, gi, go):
        self.grads = go[0]

    def remove(self):
        for h in self._h:
            h.remove()

class GradCAMViT(_GradCAM):
    def __init__(self, model, target_layer=None):
        super().__init__(model, target_layer or model.blocks[-1].norm1)

    def __call__(self, x, class_idx=None):
        self.model.eval()
        out = self.model(x)
        class_idx = out.argmax(1).item() if class_idx is None else class_idx
        self.model.zero_grad(set_to_none=True)
        out[0, class_idx].backward()
        acts, grads = self.acts[0][1:], self.grads[0][1:] # drop CLS token
        cam = (acts * grads.mean(0)).sum(-1).clamp(min=0)
        s = int(cam.shape[0] ** 0.5)
        cam = cam.reshape(s, s).detach().cpu().numpy()
        return _norm_resize(cam, x.shape[-1]), class_idx

class GradCAMResNet(_GradCAM):
    def __init__(self, model, target_layer=None):
        super().__init__(model, target_layer or model.layer4[-1])

    def __call__(self, x, class_idx=None):
        self.model.eval()
        out = self.model(x)
        class_idx = out.argmax(1).item() if class_idx is None else class_idx
        self.model.zero_grad(set_to_none=True)
        out[0, class_idx].backward()
        acts, grads = self.acts[0], self.grads[0] # (C,H,W)
        cam = (acts * grads.mean((1, 2))[:, None, None]).sum(0).clamp(min=0)
        return _norm_resize(cam.detach().cpu().numpy(), x.shape[-1]), class_idx

def _norm_resize(cam, size):
    cam = cam - cam.min()
    cam = cam / (cam.max() + 1e-8)
    return cv2.resize(cam, (size, size))

def make_gradcam(model, model_name):
    if model_name == "resnet50":
        return GradCAMResNet(model)
    if model_name in ("vit_b", "vit_l"):
        return GradCAMViT(model)

```

```
logger.warning(f"Grad-CAM not wired for '{model_name}'; returning None")
return None
```

```
def overlay_heatmap(image_rgb, cam, alpha=0.5):
    heat = cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET)
    heat = cv2.cvtColor(heat, cv2.COLOR_BGR2RGB)
    return cv2.addWeighted(np.asarray(image_rgb, dtype=np.uint8), 1 - alpha, heat, alpha, 0)
```

```
def denormalize(tensor):
    '''CHW normalized tensor -> HxWx3 uint8 RGB for visualization / overlay.'''
    mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
    std = torch.tensor(IMAGENET_STD).view(3, 1, 1)
    img = (tensor.detach().cpu() * std + mean).clamp(0, 1)
    return (img.permute(1, 2, 0).numpy() * 255).astype(np.uint8)
```

```
def save_checkpoint(state, path):
    path = Path(path); path.parent.mkdir(parents=True, exist_ok=True)
    tmp = path.with_suffix(".tmp")
    torch.save(state, tmp)
    tmp.replace(path) # atomic write -> survives a killed kernel mid-save
```

```
def load_checkpoint(path, map_location="cpu"):
    return torch.load(path, map_location=map_location, weights_only=False)
```

```
def raw_module(model):
    '''Unwrap torch.compile so checkpoints have clean (prefix-free) state_dict keys.'''
    return getattr(model, "_orig_mod", model)
```

```
print("shared.ipynb loaded: Phase A + B + helpers ready.")
```

00 — Environment Check

00 ? Environment check (AMD MI300X / ROCm)

Run ****first**** in each daily GPU window. Verifies ROCm + bf16, installs deps, logs into W&B, and sets `PERSIST\_DIR` (the mounted folder where data + checkpoints survive between sessions).

```
# Install dependencies (torch/ROCM already in the AMD image). Safe to re-run.
# Comment out once the persistent env has them.
%pip install -q albumentations torchstain scikit-image timm transformers accelerate wandb gradio opencv-
python-headless seaborn scikit-learn
print("deps installed")
```

```
import torch, platform
print("Python      :", platform.python_version())
print("PyTorch     :", torch.__version__)
print("ROCM/HIP     :", getattr(torch.version, "hip", None))
print("CUDA build  :", torch.version.cuda)
cuda = torch.cuda.is_available()
print("GPU available:", cuda)
if cuda:
    print("Device      :", torch.cuda.get_device_name(0))
    print("bf16 native :", torch.cuda.is_bf16_supported())
    free, total = torch.cuda.mem_get_info()
    print(f"HBM          : {total/1e9:.0f} GB total, {free/1e9:.0f} GB free")
else:
    print("WARNING: no GPU visible ? fine for authoring, but training needs the MI300X session.")
```

```
import os
from pathlib import Path

# Point this at the AMD *persistent / mounted* folder so data + checkpoints survive restarts.
PERSIST_DIR = Path(os.environ.get("PERSIST_DIR", "/workspace/shared/ft004"))
PERSIST_DIR.mkdir(parents=True, exist_ok=True)
for sub in ("data", "checkpoints", "outputs"):
    (PERSIST_DIR / sub).mkdir(exist_ok=True)
os.environ["PERSIST_DIR"] = str(PERSIST_DIR)
print("PERSIST_DIR =", PERSIST_DIR)
print("Free disk (GB):", round(__import__('shutil').disk_usage(PERSIST_DIR).free / 1e9, 1))
```

```
# Weights & Biases login. Needs an API key on this machine.
# If network egress is blocked, switch to offline mode instead:
# import os; os.environ["WANDB_MODE"] = "offline"
import wandb
try:
    wandb.login()          # prompts for key, or uses WANDB_API_KEY env var
    print("wandb: logged in")
except Exception as e:
    os.environ["WANDB_MODE"] = "offline"
    print(f"wandb login failed ({e}); falling back to WANDB_MODE=offline")
```

01 — Data Preparation

01 ? Data: download, EDA, augmentation preview

Downloads NCT-CRC-HE-100K (train) + CRC-VAL-HE-7K (val) into `PERSIST\_DIR/data` (once), shows the class distribution, and previews the Albumentations pipeline incl. `HEDStainJitter`.

```
# Locate shared.ipynb regardless of the kernel's working directory, then run it.
from pathlib import Path as _P
_sh = next((p for p in [_P.cwd() / "shared.ipynb",
                        _P("/workspace/shared/ft004/shared.ipynb")] if p.exists()), None)
if _sh is None:
    _hits = list(_P.cwd().rglob("shared.ipynb")) or list(_P("/workspace").rglob("shared.ipynb"))
    _sh = _hits[0] if _hits else None
assert _sh, "shared.ipynb not found - keep it beside the notebooks or in /workspace/shared/ft004"
print("running", _sh)
get_ipython().run_line_magic("run", str(_sh))
import os
from pathlib import Path
PERSIST_DIR = Path(os.environ.get("PERSIST_DIR", "/workspace/shared/ft004"))
set_seed(42)
```

```
# ~10 GB download on first run; cached thereafter. Resumes are no-ops.
paths = ensure_dataset(PERSIST_DIR)
TRAIN_DIR = paths["NCT-CRC-HE-100K"]
VAL_DIR = paths["CRC-VAL-HE-7K"]
print("train:", TRAIN_DIR)
print("val  :", VAL_DIR)
```

```
# Class distribution
import matplotlib.pyplot as plt
train_ds = PathologyPatchDataset(TRAIN_DIR, transform=None)
val_ds = PathologyPatchDataset(VAL_DIR, classes=train_ds.classes, transform=None)

from collections import Counter
ctr = Counter(lbl for _, lbl in train_ds.samples)
counts = [ctr[i] for i in range(len(train_ds.classes))]
plt.figure(figsize=(10, 4))
plt.bar(train_ds.classes, counts, color="#4C78A8")
plt.title(f"NCT-CRC-HE-100K class distribution (train={len(train_ds)}, val={len(val_ds)})")
plt.ylabel("patches"); plt.tight_layout(); plt.show()
print({c: ctr[i] for i, c in enumerate(train_ds.classes)})
```

```

# Augmentation preview: original vs train-aug (with stain) vs strong OOD shift
import numpy as np, matplotlib.pyplot as plt
from PIL import Image

train_aug = build_train_aug(use_stain_aug=True, theta=0.05)
ood_aug = build_ood_aug(theta=0.15)

idxs = [next(i for i, (_, l) in enumerate(train_ds.samples) if l == k)
         for k in range(len(train_ds.classes))]

fig, axes = plt.subplots(3, len(idxs), figsize=(2.0 * len(idxs), 6.5))
for col, i in enumerate(idxs):
    path, lbl = train_ds.samples[i]
    img = np.array(Image.open(path).convert("RGB"))
    aug = denormalize(train_aug(image=img)["image"])
    ood = denormalize(ood_aug(image=img)["image"])
    for row, (im, tag) in enumerate([(img, "orig"), (aug, "train-aug"), (ood, "OOD shift")]):
        axes[row, col].imshow(im); axes[row, col].axis("off")
        if row == 0:
            axes[row, col].set_title(train_ds.classes[lbl], fontsize=9)
        if col == 0:
            axes[row, col].set_ylabel(tag, fontsize=10)
plt.suptitle("Augmentation pipeline preview", y=1.02)
plt.tight_layout(); plt.show()

```

02 — Training

02 ? Training (native bf16, torch.compile, W&B, resume-safe)

Edit the ****CONFIG**** cell to pick the model and run. Checkpoints (`last.pth`/`best.pth`) and the W&B run-id persist to `PERSIST\_DIR`, so re-running after a session ends ****resumes**** automatically ? the key to spanning the three 6-hour GPU windows.

Suggested runs: Day 1 `resnet50` then `vit\_b`; Day 2 `vit\_l` (hero), plus `vit\_l` with `use\_stain\_aug=False` for the ablation, plus `phikon`.

```
# Locate shared.ipynb regardless of the kernel's working directory, then run it.
from pathlib import Path as _P
_sh = next((p for p in [_P.cwd() / "shared.ipynb",
                       _P("/workspace/shared/ft004/shared.ipynb")] if p.exists()), None)

if _sh is None:
    _hits = list(_P.cwd().rglob("shared.ipynb")) or list(_P("/workspace").rglob("shared.ipynb"))
    _sh = _hits[0] if _hits else None
assert _sh, "shared.ipynb not found - keep it beside the notebooks or in /workspace/shared/ft004"
print("running", _sh)
get_ipython().run_line_magic("run", str(_sh))
import os
from pathlib import Path
PERSIST_DIR = Path(os.environ.get("PERSIST_DIR", "/workspace/shared/ft004"))
set_seed(42)
```

```
# ===== CONFIG =====
cfg = dict(
    model="phikon",          # resnet50 | vit_b | vit_l | swin_b | phikon
    use_stain_aug=True,      # set False for the ablation counterpart
    hed_theta=0.05,
    img_size=224,
    epochs=12,
    batch_size=256,          # ViT-L on MI300X can push 512; ResNet/ViT-B 256
    eval_batch_size=512,
    lr=1e-4,                 # head LR (backbone uses lr * backbone_lr_mult)
    backbone_lr_mult=0.1,
    weight_decay=0.05,
    warmup_epochs=1,
    label_smoothing=0.1,
    num_workers=8,           # needs container --ipc=host / --shm-size; lower if bus errors
    pin_memory=True,
    prefetch_factor=4,
    use_compile=True,        # torch.compile(reduce-overhead); falls back to eager on failure
    resume=True,
)

run_name = f"{cfg['model']}_{{'stain' if cfg['use_stain_aug'] else 'nostain'}}"
ckpt_dir = PERSIST_DIR / "checkpoints" / run_name
ckpt_dir.mkdir(parents=True, exist_ok=True)
device = get_device()
amp_dtype = get_amp_dtype(device)
print(f"run={run_name} device={device} amp={amp_dtype}")
```

```

# Data
paths = ensure_dataset(PERSIST_DIR)
train_loader, val_loader, classes = make_data loaders(
    paths["NCT-CRC-HE-100K"], paths["CRC-VAL-HE-7K"], cfg)
print(f"classes={classes}")
print(f"train batches={len(train_loader)} val batches={len(val_loader)}")

# Model, optimizer, scheduler
from torch.optim.lr_scheduler import LinearLR, CosineAnnealingLR, SequentialLR

model = build_model(cfg, num_classes=len(classes)).to(device)
optimizer = torch.optim.AdamW(get_param_groups(model, cfg))
warmup = LinearLR(optimizer, start_factor=0.1, total_iters=max(1, cfg["warmup_epochs"]))
cosine = CosineAnnealingLR(optimizer, T_max=max(1, cfg["epochs"] - cfg["warmup_epochs"]),
                           eta_min=cfg["lr"] * 0.01)
scheduler = SequentialLR(optimizer, [warmup, cosine], milestones=[cfg["warmup_epochs"]])
criterion = nn.CrossEntropyLoss(label_smoothing=cfg["label_smoothing"])

# fp16 (non-MI300X) needs a GradScaler; bf16 on MI300X does NOT.
scaler = torch.cuda.amp.GradScaler() if amp_dtype == torch.float16 else None

if cfg["use_compile"] and device.type == "cuda":
    try:
        # "reduce-overhead" (CUDA graphs) corrupts BatchNorm running stats on ROCm,
        # causing immediate NaN loss for conv nets; use plain compile for those.
        has_bn = any(isinstance(m, nn.BatchNorm2d) for m in model.modules())
        compile_mode = "default" if has_bn else "reduce-overhead"
        model = torch.compile(model, mode=compile_mode)
        print(f"torch.compile: enabled ({compile_mode})")
    except Exception as e:
        print(f"torch.compile failed -> eager ({e})")

# Resume from last.pth if present (atomic checkpoints written every epoch)
import wandb
start_epoch, best_f1 = 0, 0.0
last_path = ckpt_dir / "last.pth"
if cfg["resume"] and last_path.exists():
    ck = load_checkpoint(last_path)
    raw_module(model).load_state_dict(ck["model"])
    optimizer.load_state_dict(ck["optimizer"])
    scheduler.load_state_dict(ck["scheduler"])
    start_epoch = ck["epoch"] + 1
    best_f1 = ck.get("best_metric", 0.0)
    logger.info(f"Resumed {run_name} from epoch {start_epoch} (best_f1={best_f1:.2f})")

# Persist W&B run id so a restarted session continues the SAME run
run_id_file = ckpt_dir / "wandb_run_id.txt"
run_id = run_id_file.read_text().strip() if run_id_file.exists() else wandb.util.generate_id()
run_id_file.write_text(run_id)
wandb.init(project="finetuning_004", id=run_id, resume="allow", name=run_name, config=cfg)

```

```

def train_one_epoch(model, loader, optimizer, criterion, device, amp_dtype, scaler, epoch):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    use_amp = device.type == "cuda"
    t0 = time.time()
    for step, (x, y) in enumerate(loader):
        x = x.to(device, non_blocking=True); y = y.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        if use_amp:
            with torch.autocast(device_type="cuda", dtype=amp_dtype):
                out = model(x)
                loss = criterion(out.float(), y)
        else:
            out = model(x); loss = criterion(out, y)
        if scaler is not None:
            scaler.scale(loss).backward(); scaler.step(optimizer); scaler.update()
        else:
            loss.backward(); optimizer.step()
        total_loss += loss.item() * y.size(0); total += y.size(0)
        correct += (out.argmax(1) == y).sum().item()
        if step % 50 == 0:
            ips = total / max(1e-6, time.time() - t0)
            logger.info(f"ep{epoch} step{step}/{len(loader)} loss {loss.item():.4f} "
                        f"acc {100*correct/total:.1f}% {ips:.0f} img/s")
    return total_loss / total, 100 * correct / total

```

```

# Training loop ? eval every epoch, log to W&B, save best + last (resume-safe)
for epoch in range(start_epoch, cfg["epochs"]):
    logger.info(f"==== epoch {epoch+1}/{cfg['epochs']} =====")
    try:
        tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, criterion,
                                           device, amp_dtype, scaler, epoch)

        scheduler.step()
        preds, labels = run_inference(model, val_loader, device, amp_dtype)
        m = compute_metrics(labels, preds, classes)
        logger.info(f"val acc {m['accuracy']:.2f}% macro-F1 {m['macro_f1']:.2f}%")

        wandb.log({"epoch": epoch, "train/loss": tr_loss, "train/acc": tr_acc,
                  "val/acc": m["accuracy"], "val/macro_f1": m["macro_f1"],
                  "lr": optimizer.param_groups[0]["lr"],
                  **{f"val/f1_{c}": v for c, v in m["per_class_f1"].items()}})
    try:
        wandb.log({"val/confusion": wandb.plot.confusion_matrix(
            probs=None, y_true=labels.tolist(), preds=preds.tolist(),
            class_names=classes)})
    except Exception as e:
        logger.warning(f"wandb confusion log skipped ({e})")

    state = dict(epoch=epoch, model=raw_module(model).state_dict(),
                 optimizer=optimizer.state_dict(), scheduler=scheduler.state_dict(),
                 best_metric=best_f1, metrics=m, cfg=cfg, classes=classes)
    save_checkpoint(state, last_path)
    if m["macro_f1"] > best_f1:
        best_f1 = m["macro_f1"]; state["best_metric"] = best_f1
        save_checkpoint(state, ckpt_dir / "best.pth")
        logger.info(f"*** new best macro-F1 {best_f1:.2f}% -> best.pth ***")
    except Exception as e:
        logger.exception(f"epoch {epoch} failed; last.pth retained for resume ({e})")
        raise

logger.info(f"DONE {run_name}: best macro-F1 {best_f1:.2f}%")
wandb.finish()

```

03 — Evaluation & Explainability

03 ? Evaluation, OOD stain ablation & Grad-CAM

Loads a trained checkpoint and produces: accuracy / **macro-F1** / per-class F1, a **confusion matrix** PNG, the **OOD stain-shift ablation** (the "15.7% ? 1.7%" story), and **Grad-CAM** overlays per class.

```
# Locate shared.ipynb regardless of the kernel's working directory, then run it.
from pathlib import Path as _P
_sh = next((p for p in [_P.cwd() / "shared.ipynb",
                        _P("/workspace/shared/ft004/shared.ipynb")] if p.exists()), None)

if _sh is None:
    _hits = list(_P.cwd().rglob("shared.ipynb")) or list(_P("/workspace").rglob("shared.ipynb"))
    _sh = _hits[0] if _hits else None
assert _sh, "shared.ipynb not found - keep it beside the notebooks or in /workspace/shared/ft004"
print("running", _sh)
get_ipython().run_line_magic("run", str(_sh))
import os
from pathlib import Path
PERSIST_DIR = Path(os.environ.get("PERSIST_DIR", "/workspace/shared/ft004"))
OUT = PERSIST_DIR / "outputs"; OUT.mkdir(parents=True, exist_ok=True)
device = get_device(); amp_dtype = get_amp_dtype(device)

RUN_NAME = "phikon_stain" # which checkpoint dir under PERSIST_DIR/checkpoints
ckpt_path = PERSIST_DIR / "checkpoints" / RUN_NAME / "best.pth"
ck = load_checkpoint(ckpt_path)
cfg, classes = ck["cfg"], ck["classes"]
print("loaded", ckpt_path, "| best macro-F1", round(ck.get("best_metric", 0), 2))
```

```
model = build_model(cfg, num_classes=len(classes)).to(device)
raw_module(model).load_state_dict(ck["model"])
model.eval()
paths = ensure_dataset(PERSIST_DIR)
VAL_DIR = paths["CRC-VAL-HE-7K"]
```

```
# In-distribution metrics + confusion matrix
val_ds = PathologyPatchDataset(VAL_DIR, classes=classes, transform=build_eval_aug(cfg["img_size"]))
val_loader = DataLoader(val_ds, batch_size=cfg["eval_batch_size"], shuffle=False,
                        num_workers=cfg["num_workers"], pin_memory=True)
preds, labels = run_inference(model, val_loader, device, amp_dtype)
m = compute_metrics(labels, preds, classes)
print(f"Accuracy {m['accuracy']:.2f}% | Macro-F1 {m['macro_f1']:.2f}%")
print(classification_report(labels, preds, target_names=classes, digits=3))
plot_confusion_matrix(labels, preds, classes, save_path=str(OUT / f"{RUN_NAME}_confusion.png"))
```

```

# OOD stain-shift ablation: same val set, strong synthetic stain shift
ood_ds = PathologyPatchDataset(VAL_DIR, classes=classes, transform=build_ood_aug(cfg["img_size"],
    theta=0.15))
ood_loader = DataLoader(ood_ds, batch_size=cfg["eval_batch_size"], shuffle=False,
    num_workers=cfg["num_workers"], pin_memory=True)
op, ol = run_inference(model, ood_loader, device, amp_dtype)
om = compute_metrics(ol, op, classes)
drop = m["accuracy"] - om["accuracy"]
print(f"In-dist acc {m['accuracy']:.2f}% -> OOD acc {om['accuracy']:.2f}% "
    f"(drop {drop:.2f} pts)")
import json
json.dump({"run": RUN_NAME, "in_dist": m, "ood": om, "ood_drop": drop},
    open(OUT / f"{RUN_NAME}_metrics.json", "w"), indent=2)
print("Run this for the stain-aug AND no-stain-aug checkpoints to make the ablation chart.")

# Grad-CAM: one example per class, saved as overlay PNGs
import numpy as np, matplotlib.pyplot as plt
cam = make_gradcam(raw_module(model), cfg["model"])
if cam is None:
    print(f"Grad-CAM not available for {cfg['model']} (use a timm vit/resnet checkpoint).")
else:
    eval_aug = build_eval_aug(cfg["img_size"])
    fig, axes = plt.subplots(2, len(classes), figsize=(2.0 * len(classes), 4.5))
    for col, c in enumerate(classes):
        i = next(idx for idx, (_, l) in enumerate(val_ds.samples) if classes[l] == c)
        path, _ = val_ds.samples[i]
        from PIL import Image
        raw = np.array(Image.open(path).convert("RGB").resize((cfg["img_size"], cfg["img_size"])))
        x = eval_aug(image=np.array(Image.open(path).convert("RGB")))[0].unsqueeze(0).to(device)
        heat, pred = cam(x)
        ov = overlay_heatmap(raw, heat)
        axes[0, col].imshow(raw); axes[0, col].axis("off"); axes[0, col].set_title(c, fontsize=9)
        axes[1, col].imshow(ov); axes[1, col].axis("off")
        axes[1, col].set_title(f"->{classes[pred]}", fontsize=8)
    plt.suptitle("Grad-CAM (top: input, bottom: heatmap overlay)", y=1.02)
    plt.tight_layout(); plt.savefig(OUT / f"{RUN_NAME}_gradcam.png", dpi=180, bbox_inches="tight")
    plt.show()
    cam.remove()

```

04 — Interactive Demo

04 ? Gradio demo (CPU)

Interactive demo for the submission: upload an H&E patch -> predicted class + confidence + full 9-class distribution + Grad-CAM overlay. Runs on **CPU** to preserve the GPU budget.

```
# Locate shared.ipynb regardless of the kernel's working directory, then run it.
from pathlib import Path as _P
_sh = next((p for p in [_P.cwd() / "shared.ipynb",
                        _P("/workspace/shared/ft004/shared.ipynb")] if p.exists()), None)
if _sh is None:
    _hits = list(_P.cwd().rglob("shared.ipynb")) or list(_P("/workspace").rglob("shared.ipynb"))
    _sh = _hits[0] if _hits else None
assert _sh, "shared.ipynb not found - keep it beside the notebooks or in /workspace/shared/ft004"
print("running", _sh)
get_ipython().run_line_magic("run", str(_sh))
import os
from pathlib import Path
import numpy as np
from PIL import Image
PERSIST_DIR = Path(os.environ.get("PERSIST_DIR", "/workspace/shared/ft004"))

RUN_NAME = "vit_l_stain"
ck = load_checkpoint(PERSIST_DIR / "checkpoints" / RUN_NAME / "best.pth", map_location="cpu")
cfg, classes = ck["cfg"], ck["classes"]
# Training is finished and the GPU is idle, so use it here too -> Grad-CAM capture in
# seconds instead of minutes. The interactive demo below works the same on GPU or CPU.
device = get_device()
model = build_model(cfg, num_classes=len(classes)).to(device)
raw_module(model).load_state_dict(ck["model"])
model.eval()
cam = make_gradcam(raw_module(model), cfg["model"])
eval_aug = build_eval_aug(cfg["img_size"])
print("demo model ready:", cfg["model"], "on", device)
```

```
def classify(image, use_macenko=False):
    img = np.array(image.convert("RGB"))
    if use_macenko:
        try:
            img = MacenkoNormalizer(target_img=img)(img) # self-normalize as a light demo
        except Exception as e:
            logger.warning(f"Macenko skipped ({e})")
    x = eval_aug(image=img)["image"].unsqueeze(0).to(device)
    with torch.no_grad():
        probs = model(x).softmax(1)[0]
        top = int(probs.argmax())
        label_desc = f"{classes[top]} ? {CLASS_DESCRIPTIONS[classes[top]]}"
        dist = {f"{c} ({CLASS_DESCRIPTIONS[c]})": float(probs[i]) for i, c in enumerate(classes)}
        if cam is not None:
            raw = np.array(Image.fromarray(img).resize((cfg["img_size"], cfg["img_size"])))
            heat, _ = cam(x, class_idx=top)
            overlay = Image.fromarray(overlay_heatmap(raw, heat))
        else:
            overlay = image
    return label_desc, dist, overlay
```

```

# Offline demo capture: run classify() on one example per class, save a grid + predictions JSON
# so the demo's behaviour can be reviewed without a live Gradio server.
import json as _json
import matplotlib.pyplot as plt

paths = ensure_dataset(PERSIST_DIR)
VAL_DIR = paths["CRC-VAL-HE-7K"]
demo_ds = PathologyPatchDataset(VAL_DIR, classes=classes, transform=None)

OUT = PERSIST_DIR / "outputs"; OUT.mkdir(parents=True, exist_ok=True)
results = []
fig, axes = plt.subplots(3, len(classes), figsize=(2.2 * len(classes), 6.5))
for col, c in enumerate(classes):
    i = next(idx for idx, (_, l) in enumerate(demo_ds.samples) if classes[l] == c)
    path, _ = demo_ds.samples[i]
    img = Image.open(path).convert("RGB")
    label_desc, dist, overlay = classify(img)
    pred_class = label_desc.split(" ? ")[0]
    pred_key = next(k for k in dist if k.startswith(pred_class))
    results.append({"true": c, "pred": pred_class, "confidence": float(dist[pred_key]), "file": str(path)})
    axes[0, col].imshow(img); axes[0, col].axis("off"); axes[0, col].set_title(f"true: {c}", fontsize=9)
    axes[1, col].imshow(overlay); axes[1, col].axis("off"); axes[1, col].set_title(f"pred: {pred_class}",
    fontsize=9)
    axes[2, col].bar(range(len(classes)), [dist[k] for k in dist], color="steelblue")
    axes[2, col].set_xticks([]); axes[2, col].set_ylim(0, 1)
plt.suptitle(f"Offline demo examples ? model: {cfg['model']} ({RUN_NAME})", y=1.02)
plt.tight_layout()
plt.savefig(OUT / "demo_offline_examples.png", dpi=160, bbox_inches="tight")
plt.show()
_json.dump(results, open(OUT / "demo_offline_examples.json", "w"), indent=2)
for r in results:
    print(f"{r['true']:>5} -> {r['pred']:<5} (conf {r['confidence']:.3f})")

```

```

_ensure("gradio")
import gradio as gr
import time

with gr.Blocks(title="FINETUNING_004 ? Stain-Robust Pathology Classifier") as demo:
    gr.Markdown("# FINETUNING_004 ? Stain-Robust Pathology Classifier\n"
                "Upload a 224x224 H&E patch -> tissue class + confidence + Grad-CAM. "
                f"Model: **{cfg['model']}** fine-tuned on NCT-CRC-HE-100K (AMD MI300X).")
    with gr.Row():
        with gr.Column():
            inp = gr.Image(type="pil", label="H&E patch")
            mac = gr.Checkbox(label="Macenko normalize upload", value=False)
            btn = gr.Button("Classify", variant="primary")
        with gr.Column():
            out_label = gr.Markdown(label="Prediction")
            out_dist = gr.Label(num_top_classes=9, label="Class probabilities")
            out_cam = gr.Image(type="pil", label="Grad-CAM overlay")
    btn.click(classify, inputs=[inp, mac], outputs=[out_label, out_dist, out_cam])

# Non-blocking launch: prove the demo starts, then close it so notebook execution can
# finish (the offline example grid above is the artifact for the static demo). A prior
# session left a server bound to 8860, so pick a free port for this launch.
import socket
def _free_port(candidates):
    for p in candidates:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            s.bind(("0.0.0.0", p)); return p
        except OSError:
            continue
    finally:
        s.close()
    return candidates[-1]

port = _free_port([8860, 7860, 8861, 7861, 8862])
demo.launch(server_name="0.0.0.0", server_port=port, share=False, prevent_thread_lock=True)
time.sleep(3)
print(f"Gradio demo launched successfully on port {port} (offline examples captured above).")
demo.close()

```

05 — Visual Showcase

05 ? Visual Showcase

****FINETUNING_004 · Stain-Robust Pathology Classifier****

Three cells:

1. Load model (ViT-L/16, fine-tuned on AMD MI300X)
2. Classify three H&E patches ? Grad-CAM overlay + 9-class probability chart
3. Stain-robustness ablation ? the headline result

```

# ?? Bootstrap: load shared library + fine-tuned ViT-L/16 checkpoint ??????????
import json, os
from pathlib import Path
import numpy as np
import torch
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

_repo = Path(globals().get('__vsc_ipynb_file__', __file__ if '__file__' in dir() else '.')).parent
_shared = next((p for p in [_repo / 'shared.ipynb',
                             Path('/workspace/finetuning-004-pathology/shared.ipynb')] if p.exists()), None)
assert _shared, 'shared.ipynb not found'
_nb = json.loads(_shared.read_text())
for _cell in _nb['cells']:
    if _cell['cell_type'] == 'code':
        _src = ''.join(_cell['source'])
        _src = '\n'.join(l for l in _src.splitlines()
                          if not l.strip().startswith(('%', 'get_ipython')))
        if _src.strip():
            exec(_src, globals())

PERSIST_DIR = Path(os.environ.get('PERSIST_DIR', '/workspace/shared/ft004'))
RUN_NAME = 'vit_l_stain'
ck = load_checkpoint(PERSIST_DIR / 'checkpoints' / RUN_NAME / 'best.pth', map_location='cpu')
cfg, classes = ck['cfg'], ck['classes']
device = get_device()
model = build_model(cfg, num_classes=len(classes)).to(device)
raw_module(model).load_state_dict(ck['model'])
model.eval()
cam = make_gradcam(raw_module(model), cfg['model'])
eval_aug = build_eval_aug(cfg['img_size'])
print(f'Model ready: {cfg["model"]} on {device}')

def classify(pil_image):
    img = np.array(pil_image.convert('RGB'))
    x = eval_aug(image=img)['image'].unsqueeze(0).to(device)
    with torch.no_grad():
        probs = model(x).softmax(1)[0]
    top = int(probs.argmax())
    pred = classes[top]
    conf = float(probs[top])
    dist = {c: float(probs[i]) for i, c in enumerate(classes)}
    raw_arr = np.array(Image.fromarray(img).resize((cfg['img_size'], cfg['img_size'])))
    if cam is not None:
        heat, _ = cam(x, class_idx=top)
        overlay = Image.fromarray(overlay_heatmap(raw_arr, heat))
    else:
        overlay = pil_image
    return pred, conf, dist, overlay

```

```

# ?? Classify three representative patches ? full visual breakdown ??????????????
VAL = PERSIST_DIR / 'data' / 'CRC-VAL-HE-7K' / 'CRC-VAL-HE-7K'
DEMO_PATCHES = [
    ('TUM', 'Colorectal adenocarcinoma epithelium', VAL / 'TUM' / 'TUM-TCGA-AAHIDTWA.tif'),
    ('STR', 'Cancer-associated stroma', VAL / 'STR' / 'STR-TCGA-AAMALCER.tif'),
    ('LYM', 'Lymphocytes', VAL / 'LYM' / 'LYM-TCGA-AAWGSCHH.tif'),
]

results = []
for cls, desc, path in DEMO_PATCHES:
    img = Image.open(path).convert('RGB')
    pred, conf, dist, overlay = classify(img)
    results.append(dict(true=cls, desc=desc, img=img, overlay=overlay,
                        dist=dist, pred=pred, conf=conf))
    print(f'{cls} ? {pred} ({conf*100:.1f}%)')

# ?? Figure ??????????????????????????????????????????????????????????????
BG = '#0d1117'
CARD = '#161b22'
COLS = {'TUM': '#e05c5c', 'STR': '#f0a040', 'LYM': '#3ecf8e',
        'ADI': '#60a5fa', 'BACK': '#94a3b8', 'DEB': '#a78bfa',
        'MUC': '#34d399', 'MUS': '#fbbf24', 'NORM': '#4ade80'}

fig = plt.figure(figsize=(17, 13), facecolor=BG)
fig.suptitle(
    'FINETUNING_004 · Stain-Robust Pathology Classifier · ViT-L/16 on AMD MI300X',
    color='white', fontsize=13, fontweight='bold', y=0.995
)

outer = gridspec.GridSpec(1, 3, figure=fig, wspace=0.08,
                           left=0.03, right=0.97, top=0.96, bottom=0.02)

for col, r in enumerate(results):
    inner = gridspec.GridSpecFromSubplotSpec(
        3, 1, subplot_spec=outer[col],
        hspace=0.32, height_ratios=[3.2, 3.2, 2.8]
    )
    accent = COLS.get(r['true'], '#ffffff')
    ok = r['pred'] == r['true']
    tick = COLS.get(r['pred'], '#00ff88') if ok else '#ff5555'

    # ?? row 0: original patch ??????????????????????????????????????????????
    ax0 = fig.add_subplot(inner[0])
    ax0.imshow(r['img'])
    ax0.axis('off')
    ax0.set_facecolor(CARD)
    for sp in ax0.spines.values():
        sp.set_visible(True); sp.set_edgecolor(accent); sp.set_linewidth(2.5)
    ax0.set_title(
        f"{r['true']} · {r['desc']}",
        color=accent, fontsize=10, fontweight='bold', pad=7
    )
    ax0.text(0.02, 0.04, 'H&E patch · 224 × 224', transform=ax0.transAxes,
             color='#8b949e', fontsize=7.5, va='bottom')

    # ?? row 1: Grad-CAM overlay ??????????????????????????????????????????????
    ax1 = fig.add_subplot(inner[1])
    ax1.imshow(r['overlay'])
    ax1.axis('off')
    ax1.set_facecolor(CARD)
    for sp in ax1.spines.values():
        sp.set_visible(True); sp.set_edgecolor(tick); sp.set_linewidth(2.5)

```

```

ax1.set_title(
    f"{'?' if ok else ''} {r['pred']} · {r['conf']*100:.1f}% confidence",
    color=tick, fontsize=10, fontweight='bold', pad=7
)
ax1.text(0.02, 0.04, 'Grad-CAM overlay', transform=ax1.transAxes,
        color='#8b949e', fontsize=7.5, va='bottom')

# ?? row 2: probability bar chart ?????????????????????????????????????????
ax2 = fig.add_subplot(inner[2])
ax2.set_facecolor(CARD)
labels = list(r['dist'].keys())
probs = list(r['dist'].values())
order = sorted(range(len(probs)), key=lambda i: probs[i])
s_labels = [labels[i] for i in order]
s_probs = [probs[i] for i in order]
bar_cols = [COLS.get(labels[i], '#60a5fa') if labels[i] == r['pred']
            else '#2d333b' for i in order]
bars = ax2.barh(s_labels, s_probs, color=bar_cols, edgecolor='none', height=0.72)
for bar, prob, lbl in zip(bars, s_probs, s_labels):
    if prob > 0.02:
        ax2.text(min(prob + 0.01, 0.97), bar.get_y() + bar.get_height() / 2,
                f'{prob*100:.1f}%', va='center', color='white', fontsize=7.5,
                fontweight='bold' if lbl == r['pred'] else 'normal')
ax2.set_xlim(0, 1.05)
ax2.set_xlabel('Probability', color='#8b949e', fontsize=8)
ax2.tick_params(colors='#8b949e', labelsize=7.5)
for sp in ['top', 'right']:
    ax2.spines[sp].set_visible(False)
for sp in ['bottom', 'left']:
    ax2.spines[sp].set_color('#30363d')
ax2.set_title('9-class probability distribution', color='#8b949e', fontsize=8, pad=5)

out_path = PERSIST_DIR / 'outputs' / 'showcase.png'
plt.savefig(out_path, dpi=180, bbox_inches='tight', facecolor=BG)
plt.show()
print(f'Saved ? {out_path}')

```

```

# ?? Headline result: stain-robustness ablation ?????????????????????????????????
import json as _json

stain = _json.loads((PERSIST_DIR / 'outputs' / 'vit_l_stain_metrics.json').read_text())
nostain = _json.loads((PERSIST_DIR / 'outputs' / 'vit_l_nostain_metrics.json').read_text())

s_clean, s_ood = stain['in_dist']['accuracy'], stain['ood']['accuracy']
n_clean, n_ood = nostain['in_dist']['accuracy'], nostain['ood']['accuracy']
s_drop, n_drop = stain['ood_drop'], nostain['ood_drop']

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), facecolor=BG)
fig.suptitle(
    'Stain-Robustness Ablation · ViT-L/16 · HED Augmentation vs Baseline',
    color='white', fontsize=12, fontweight='bold', y=1.01
)

# Left: grouped bar ? clean vs OOD accuracy
ax = axes[0]
ax.set_facecolor(CARD)
x = [0, 1]
width = 0.32
b1 = ax.bar([v - width/2 for v in x], [n_clean, n_ood], width,
            label='No stain aug (baseline)', color='#e05c5c', alpha=0.85, edgecolor='none')
b2 = ax.bar([v + width/2 for v in x], [s_clean, s_ood], width,
            label='HED stain aug (ours)', color='#3ecf8e', alpha=0.85, edgecolor='none')
for bar in list(b1) + list(b2):
    h = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, h + 0.5, f'{h:.1f}%',
            ha='center', va='bottom', color='white', fontsize=9, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(['Clean validation\n(in-distribution)', 'Stain-shifted\n(out-of-distribution)'],
                    color='#8b949e', fontsize=10)
ax.set_ylabel('Accuracy (%)', color='#8b949e', fontsize=10)
ax.set_ylim(0, 108)
ax.tick_params(colors='#8b949e')
ax.legend(facecolor='#0d1117', edgecolor='#30363d', labelcolor='white', fontsize=9)
ax.set_title('Accuracy: clean vs stain-shifted', color='white', fontsize=10, pad=8)
for sp in ['top', 'right']:
    ax.spines[sp].set_visible(False)
for sp in ['bottom', 'left']:
    ax.spines[sp].set_color('#30363d')

# Right: accuracy drop comparison
ax2 = axes[1]
ax2.set_facecolor(CARD)
bars2 = ax2.bar(['No stain aug\n(baseline)', 'HED stain aug\n(ours)'],
                [n_drop, s_drop],
                color=['#e05c5c', '#3ecf8e'], alpha=0.85, edgecolor='none', width=0.45)
for bar, val in zip(bars2, [n_drop, s_drop]):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
            f'{val:.1f} pp', ha='center', va='bottom',
            color='white', fontsize=12, fontweight='bold')
ax2.set_ylabel('Accuracy drop (percentage points)', color='#8b949e', fontsize=10)
ax2.set_ylim(0, 52)
ax2.tick_params(colors='#8b949e')
ax2.set_title(f'Robustness gap reduced by {n_drop/s_drop:.1f}x', color='white', fontsize=10, pad=8)
ax2.text(0.5, 0.55,
        f'{n_drop/s_drop:.1f}x smaller\ndrop with HED aug',
        transform=ax2.transAxes, ha='center', color='#3ecf8e',
        fontsize=14, fontweight='bold')
for sp in ['top', 'right']:
    ax2.spines[sp].set_visible(False)

```

```
for sp in ['bottom', 'left']:
    ax2.spines[sp].set_color('#30363d')

plt.tight_layout()
out_path2 = PERSIST_DIR / 'outputs' / 'showcase_robustness.png'
plt.savefig(out_path2, dpi=180, bbox_inches='tight', facecolor=BG)
plt.show()
print(f'Stain drop: baseline {n_drop:.1f} pp ? ours {s_drop:.1f} pp ({n_drop/s_drop:.1f}× reduction)')
print(f'Saved ? {out_path2}')
```